


**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**

*Bhartiya Vidya Bhavan's*
*Sardar Patel Institute of Technology (S.P.I.T.)*

---

# A Project Report on
# RoomSage

*An AI-Powered Interior Redesign and 3D Room Planning Framework with Catalog-Grounded, Budget-Aware Furniture Placement*

---

Submitted in partial fulfillment of the requirements for the degree of **Bachelor of Engineering (Computer Science and Engineering)**

## Submitted by:

| | |
|---|---|
| **Miheera Nikam** | UID: 2023800069 |
| **Sanvida Gokhale** | UID: 2023800026 |
| **Pratham Masulkar** | UID: 2023800056 |

## Under the guidance of:

Dr. Shanti Swami
*Department of Computer Science and Engineering*

Academic Year 2026–2027

# ACKNOWLEDGEMENTS

We would like to express our sincere gratitude to our project guide, Dr. Shanti Swami, for the guidance, encouragement, and constructive feedback offered throughout the development of RoomSage. Their insights into system design and applied computer vision shaped the direction of this work.

We extend our thanks to the faculty members of the Department of Computer Science and Engineering for their support and for sharing the knowledge that broadened our understanding of the subject.

We also acknowledge the open-source communities and providers behind React, Vite, Three.js, FastAPI, SQLAlchemy, PostgreSQL, Celery, Redis, MinIO, PyTorch, Hugging Face Diffusers and Transformers, OpenCV, COLMAP, Nerfstudio, and the Meshy and Google Gemini APIs, whose tools formed the building blocks of RoomSage. We also thank Kaggle for the free T4 GPU environment used to run our pipelines.

Finally, we thank our families and friends for their unwavering support.

Miheera Nikam
Sanvida Gokhale
Pratham Masulkar

# ABSTRACT

Interior redesign suffers from two gaps. The first is a *visualization gap*: generic text-to-image tools hallucinate walls, doors, and windows, so their output cannot be trusted as a representation of a user's real room. The second is an *execution gap*: even a good render does not tell the user what to buy, from where, and for how much within a fixed budget in local currency.

This project presents **RoomSage**, a full-stack web application that closes both gaps. A user uploads one to three photographs of a room, selects a design style and a colour palette, and sets a budget. The system segments walls and floors, recolours them, and uses a Stable Diffusion v1.5 inpainting model to place catalog furniture only inside the user's real photograph, so the room's geometry is preserved. Every placed item is a real catalog product with a price, and a budget engine keeps the running total within the cap, shows only styles for which a complete essential set (sofa, chair, coffee table, lighting, rug) exists, and reports items that could not be placed. In parallel, a 3D room preview is assembled in the browser with Three.js, with furniture meshes generated through the Meshy API.

RoomSage is built with React 19, Vite, and Three.js on the frontend; FastAPI, SQLAlchemy, and PostgreSQL on the backend; Celery and Redis for asynchronous jobs; and MinIO for S3-compatible storage. The GPU image worker uses CUDA-enabled PyTorch, Hugging Face Transformers/Diffusers, OpenCV, and Pillow, and runs on a Kaggle T4 notebook exposed through Cloudflare. Gemini, Meshy, COLMAP, and Nerfstudio support specific AI and 3D workflows.

**Keywords:** Interior Design Automation, Stable Diffusion Inpainting, Semantic Segmentation, 3D Room Preview, Budget-Constrained Planning, Catalog Grounding, Three.js, FastAPI.

# Contents


**1 Introduction** **1**

1.1 Background . . . . . 1

1.2 Problem Statement . . . . . 1

1.3 Objectives . . . . . 2

1.4 Scope of the Project . . . . . 2

1.5 Organization of the Report . . . . . 2

**2 Literature Survey** **3**

2.1 3D Gaussian Splatting and Indoor Reconstruction . . . . . 3

2.2 Spatially Conditioned Diffusion . . . . . 3

2.3 Diffusion Inpainting . . . . . 4

2.4 Text-Driven 3D Editing and Single-Image 3D Generation . . . . . 4

2.5 Segmentation . . . . . 4

**3 System Requirements and Analysis** **5**

3.1 Functional Requirements . . . . . 5

3.2 Non-Functional Requirements . . . . . 6

3.3 Hardware Requirements . . . . . 6

3.4 Software Requirements . . . . . 7

3.5 Feasibility Study . . . . . 7

**4 System Design and Architecture** **8**

4.1 Relationship to the Proposed Framework . . . . . 8

4.2 High-Level Architecture . . . . . 8

4.3 Backend Component Design . . . . . 9

4.4 Frontend Component Design . . . . . 9
4.5 Data Flow . . . . . 10
4.6 Database Design . . . . . 10
4.7 Security Considerations . . . . . 10

**5 Implementation** **11**
5.1 Authentication and Project Management . . . . . 11
5.2 Photo Ingestion and Coverage Check . . . . . 11
5.3 Wall and Floor Segmentation . . . . . 11
5.4 Style and Palette Selection . . . . . 11
5.5 2D Generation with Stable Diffusion Inpainting . . . . . 12
5.6 Catalog and Budget Engine . . . . . 12
5.7 3D Room Preview with Meshy . . . . . 13
5.8 Frontend Implementation . . . . . 13
5.9 Asynchronous Job Handling . . . . . 13

**6 Testing and Results** **14**
6.1 Test Environment . . . . . 14
6.2 User Workflow Results . . . . . 14
6.3 Segmentation and Recolouring Results . . . . . 17
6.4 Furniture Placement Results . . . . . 18
6.5 3D Room Preview . . . . . 20
6.6 Budget Behaviour . . . . . 20
6.7 Discussion . . . . . 21

**7 Conclusion and Future Work** **22**
7.1 Conclusion . . . . . 22
7.2 Future Work . . . . . 22

**A Installation and Setup Guide** **23**
A.1 Prerequisites . . . . . 23
A.2 Backend and Workers . . . . . 23

A.3 Frontend . . . . . 23
A.4 Image Worker (Kaggle T4) . . . . . 23
A.5 Environment Variables . . . . . 24

# List of Figures


4.1 High-level architecture of RoomSage. . . . . 9

4.2 Data flow for 2D redesign generation. . . . . 10

6.1 Sign-in screen. Rooms and designs stay private to the user's account. . . . . 14

6.2 Photo upload with Corner A, Opposite corner, and Window wall uploaded; the coverage panel reports 2D redesign ready and shows 3D-model readiness. . . . 15

6.3 Style selection. Only styles with a complete essential catalog set are shown, with category counts and minimum budget. . . . . 15

6.4 Palette selection: Clean Neutrals, Warm Earth, Misty Greys, Botanical Green, Deep Ocean, and Cream & Wood. . . . . 16

6.5 Intermediate pipeline results for six sample photographs. Columns: input, wall mask, floor mask, restyled image, and final image. The masks isolate walls and floors while excluding windows and doors, so the restyling leaves them intact. . 17

6.6 Design workspace *before* the diffusion model was trained for the task: furniture is composited as cut-out objects. Items are small, the rug is rendered as a flat slab against the wall, and two items are flagged (pendant lamp not visible, lint roller not placed). The budget is Rs. 64,439 of Rs. 65,440. . . . . 18

6.7 A 2D generation result for the living-room template: sofa, coffee table, and rug are placed on the floor while the window, walls, and perspective are preserved. . 18

6.8 Final product of 2D generation (Industrial style, Clean Neutrals palette): the sofa, lounge chair, glass coffee table, and woven rug are lit and scaled consistently with the room, the wall colour follows the palette, and wall art is added. . 19

6.9 3D room preview rendered in the browser: walls, window, door, floor, and a Meshy-generated furniture mesh placed in the room. . . . . 20

# List of Tables

2.1 Comparison of RoomSage with representative approaches . . . . . 4

3.1 Hardware requirements . . . . . 6

3.2 Software requirements . . . . . 7

4.1 Proposed framework versus implemented system . . . . . 8

5.1 Style catalog coverage and minimum budget (from the running system) . . . . 12

6.1 Test environment configuration . . . . . 14

6.2 Itemised plan for Industrial / Clean Neutrals . . . . . 20

A.1 Environment variables . . . . . 24

# Chapter 1

# Introduction

## 1.1 Background

Residential renovation is one of the largest discretionary expenses a household makes, and also one of the hardest to visualise. Homeowners, tenants, interior architects, and real-estate stakeholders routinely ask the same question: *what would this room look like if it were furnished in a particular style, and what would that cost?*

Generative AI has made striking room imagery trivially easy to produce. A text-to-image model can render a "warm Scandinavian study" in seconds. However, the same freedom that makes these models creative also makes them unreliable for planning: they redraw the room rather than redesign it. At the same time, traditional 3D tools demand modelling skills, and e-commerce sites show furniture in isolation, with no link to how it would look in the user's own space.

## 1.2 Problem Statement

Two persistent gaps limit existing tools:

1. **Visualization gap.** Unconstrained 2D diffusion models distort wall dimensions, fabricate doors and windows, alter the camera perspective, and ignore existing geometry. Their output is inspirational but not a valid depiction of the user's actual room.
2. **Execution gap.** Even when a realistic render exists, there is no automatic bridge from the picture to a purchasable list of products with regional pricing and a hard budget cap. Users must identify, price, and tally items by hand.

## 1.3 Objectives

1. To build a web application that accepts one to three photographs of a real room and produces a redesigned image that preserves the room's geometry.
2. To restrict edits to the room's own surfaces by segmenting walls and floors and by inpainting furniture only into masked regions.
3. To let users choose a design style and colour palette, with only styles that have a complete essential catalog set offered.
4. To ground every placed furniture item in a structured product catalog with prices in Indian Rupees and to enforce a budget cap.
5. To provide a browser-based 3D room preview with furniture meshes generated from catalog items.
6. To support redesign history (versions), swapping and removal of items, and regeneration, while keeping each user's rooms private to their account.
7. To engineer the pipeline to run on a modest consumer-grade GPU (a Kaggle T4).

## 1.4 Scope of the Project

**In scope:** single-room redesign from user photographs; wall and floor recolouring; catalog-based placement of sofas, chairs, coffee tables, lighting, rugs, and wall colour; budget tracking; 3D room preview; user authentication and private storage.

**Out of scope:** structural alterations, electrical and plumbing planning, live e-commerce checkout, augmented-reality walkthroughs, and multi-user collaborative editing. These are listed as future work.

## 1.5 Organization of the Report

Chapter 2 reviews related work. Chapter 3 states requirements and feasibility. Chapter 4 presents architecture, data flow, and database design. Chapter 5 describes the implementation of each module. Chapter 6 reports results with screenshots of the working system. Chapter 7 concludes and outlines future work.

# Chapter 2

# Literature Survey

This chapter reviews work in four areas that inform RoomSage: radiance-field scene reconstruction, spatially conditioned diffusion, diffusion-based editing and inpainting, and single-image 3D asset generation.

## 2.1 3D Gaussian Splatting and Indoor Reconstruction

Kerbl et al. introduced 3D Gaussian Splatting (3DGS), which represents a scene with anisotropic 3D Gaussians and renders it in real time [1]. Indoor reconstruction remains difficult because of textureless walls, wide camera motion, and uncontrolled lighting. FreeSplat [4] and GaussianRoom [3] add cross-view aggregation and SDF guidance to reduce floaters. Structure-from-Motion with COLMAP [10] provides the camera poses on which such pipelines are built. These methods require many overlapping photographs and long optimisation, which motivated us to evaluate COLMAP and Nerfstudio and ultimately to favour a lighter room-preview path for the interactive product.

## 2.2 Spatially Conditioned Diffusion

ControlNet [2] conditions a frozen diffusion backbone on structural signals such as depth or edges, which reduces spatial hallucination. The proposed RoomSage paper uses depth-conditioned ControlNet with SDXL. In implementation, we adopted an *inpainting* formulation instead, where geometry is preserved by construction because pixels outside the mask are never regenerated.

## 2.3 Diffusion Inpainting

Latent diffusion models [11] perform generation in a compressed latent space, making them practical on small GPUs. The Stable Diffusion v1.5 inpainting checkpoint takes an image, a binary mask, and a prompt, and synthesises content only within the mask. This is the model used by RoomSage for furniture placement.

## 2.4 Text-Driven 3D Editing and Single-Image 3D Generation

Instruct-GS2GS [5], GaussCtrl [6], and GaussianEditor [7] edit 3D Gaussian scenes using 2D diffusion guidance. TripoSR [8] and commercial services such as Meshy generate textured 3D meshes from a single image or text prompt. RoomSage uses the Meshy API to turn catalog product images into 3D furniture meshes.

## 2.5 Segmentation

Segment Anything [9] and semantic segmentation networks provide object- and surface-level masks. RoomSage derives wall and floor masks, which are the control signal for recolouring and for restricting where furniture may be placed.

Table 2.1: Comparison of RoomSage with representative approaches

| System | Preserves room | Real catalog | Budget cap | 3D preview |
|---|---|---|---|---|
| Text-to-image (generic) | × | × | × | × |
| ControlNet + SDXL | ✓ | × | × | × |
| Instruct-GS2GS | ✓ | × | × | ✓ |
| Generic CAD/render tools | ✓ | × | × | ✓ |
| **RoomSage** | ✓ | ✓ | ✓ | ✓ |

# Chapter 3

# System Requirements and Analysis

## 3.1 Functional Requirements

**FR1** The system shall let users register and sign in; rooms and designs shall be private to the account.

**FR2** Users shall create projects (for example “My Flat”) containing rooms (for example “Living Room”).

**FR3** The system shall accept up to four photographs per room (Corner A, Opposite corner, Window wall, Optional extra) and report coverage: one photo is sufficient for a 2D redesign, three for a 3D room preview.

**FR4** The system shall provide template photos for users without a suitable photograph.

**FR5** The system shall list only styles that have a complete essential catalog set (sofa, chair, coffee table, lighting, rug) and display each style’s minimum budget.

**FR6** The system shall offer colour palettes and apply the chosen palette to walls and floor.

**FR7** The system shall generate a redesigned image in which furniture is placed only in the room’s floor region and wall colours are recoloured.

**FR8** The system shall present an itemised list with quantities, unit prices, a running total, and the budget cap.

**FR9** The system shall flag items that are not visible or not placed, with a reason (for example, “maximum of 6 objects per design”).

**FR10** Users shall be able to swap or remove an item, regenerate the design, and view earlier versions.

**FR11** The system shall generate a 3D room preview with furniture meshes.

## 3.2 Non-Functional Requirements

**NFR1** **Fidelity:** pixels outside the placement mask shall remain those of the user's photograph.

**NFR2** **Performance:** long-running image and 3D jobs shall run asynchronously so that the interface remains responsive.

**NFR3** **Resource efficiency:** the image pipeline shall run in fp16 on a 16 GB T4 GPU.

**NFR4** **Security:** credentials shall be stored as salted hashes in PostgreSQL; assets shall be served only to their owner.

**NFR5** **Maintainability:** service-oriented backend with clear separation of API, workers, and storage.

**NFR6** **Honesty of output:** a budget total shall be derived from catalog prices, never asserted without a source.

## 3.3 Hardware Requirements

Table 3.1: Hardware requirements

| Component | Minimum | Used in this project |
|---|---|---|
| Client device | Any WebGL-capable device | Laptop, Chrome |
| Server CPU/RAM | 4 cores / 8 GB | Local workstation |
| GPU (image worker) | 16 GB VRAM class | Kaggle NVIDIA T4 |
| Network | Broadband | Broadband + Cloudflare tunnel |

## 3.4 Software Requirements

Table 3.2: Software requirements

| Layer | Technology | Purpose |
|---|---|---|
| Frontend | React 19, Vite, Three.js | UI, 3D preview |
| Backend | FastAPI, SQLAlchemy | REST API, ORM |
| Database | PostgreSQL | Users, projects, designs, catalog |
| Jobs | Celery, Redis | Asynchronous tasks |
| Storage | MinIO (S3-compatible) | Photos, masks, renders |
| Image worker | PyTorch (CUDA), Diffusers, Transformers, OpenCV, Pillow | Segmentation, inpainting |
| External AI/3D | Gemini, Meshy, COLMAP, Nerfstudio | Scene understanding, mesh generation, reconstruction experiments |
| Compute | Kaggle notebook (T4 GPU), Cloudflare | GPU execution and public endpoint |

## 3.5 Feasibility Study

**Technical.** All components are mature open-source projects or have public APIs. Stable Diffusion v1.5 inpainting in fp16 fits comfortably on a T4.

**Economic.** The stack is open-source; GPU time uses Kaggle's free tier, and the model licence (CreativeML OpenRAIL-M) permits this academic use. The only metered services are the Meshy and Gemini APIs.

**Operational.** The system is a standard web application; users need only a browser and a smartphone photograph.

# Chapter 4

# System Design and Architecture

## 4.1 Relationship to the Proposed Framework

The initial RoomSage proposal described a five-stage hybrid pipeline (COLMAP and 3DGS reconstruction, SDXL with ControlNet, Instruct-GS2GS editing, TripoSR assets, and an LLM planner). During implementation we kept its goals—spatial fidelity, catalog grounding, budget enforcement—but adjusted the technique where free-tier GPU limits and interaction latency made the original impractical. Table 4.1 summarises the differences.

Table 4.1: Proposed framework versus implemented system

| Capability | Proposed | Implemented |
|---|---|---|
| 2D redesign | SDXL + depth ControlNet | Stable Diffusion v1.5 inpainting (fp16), mask-restricted |
| 3D assets | TripoSR | Meshy API |
| Room 3D | 3DGS splat of the room | Three.js room preview built from photo coverage; COLMAP/Nerfstudio evaluated |
| Planner | LLM over catalog | Rule-based catalog and budget engine; Gemini for scene support |
| Compute | Local GPU | Kaggle T4 via Cloudflare |

## 4.2 High-Level Architecture

RoomSage follows a decoupled client–server design. The browser talks to a FastAPI backend over HTTP/JSON. Long-running work is placed on a Redis queue and executed by Celery workers; the GPU worker runs separately and is reached through a Cloudflare endpoint.

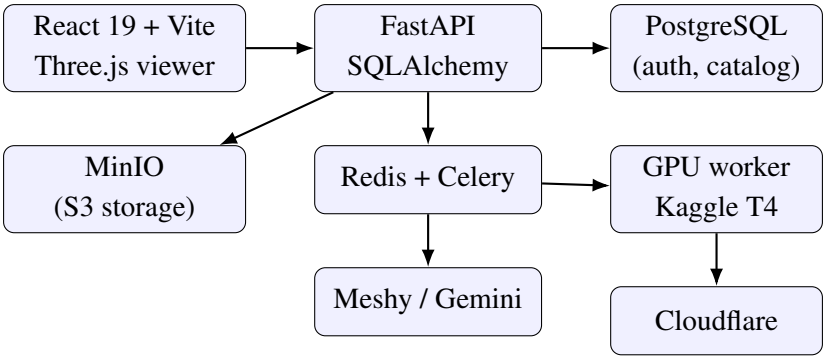



Figure 4.1: High-level architecture of RoomSage.

## 4.3 Backend Component Design

**API service (FastAPI).** Authentication, projects, rooms, photo upload, style and palette listing, design creation, versions, swap/remove, and regenerate endpoints.

**Data layer (SQLAlchemy + PostgreSQL).** Stores users with hashed credentials, projects, rooms, photos, designs and their versions, design items, and the product catalog.

**Job layer (Celery + Redis).** Image segmentation, inpainting, and mesh generation are queued so that requests return immediately and the UI can poll job status.

**Object storage (MinIO).** Holds uploaded photographs, wall and floor masks, intermediate and final renders, and generated meshes under S3-style keys.

**GPU image worker.** A CUDA-enabled PyTorch process performing segmentation, recolouring, and Stable Diffusion inpainting. It is run in a Kaggle notebook with a T4 GPU and made reachable through Cloudflare.

## 4.4 Frontend Component Design

The single-page application follows the user's flow: sign-in, projects and rooms, photo upload with coverage indicator, style selection, palette selection, and a design workspace. The workspace shows the generated image with numbered markers over placed items, a side panel with *Items*, *Edit*, and *Versions* tabs, a budget bar, and a *Regenerate* action. A Three.js viewer renders the 3D room preview.

## 4.5 Data Flow

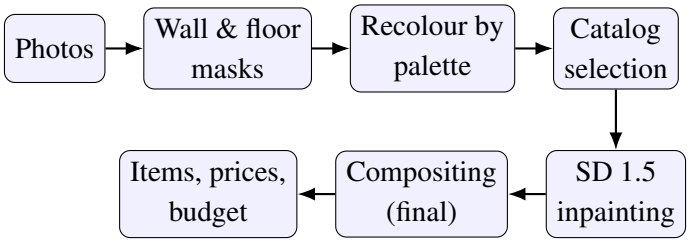



Figure 4.2: Data flow for 2D redesign generation.

## 4.6 Database Design

PostgreSQL is used for relational data. The principal entities are:

- **users**: id, email, password hash, created time.
- **projects** and **rooms**: a user owns projects; a project contains rooms with a room type (for example, living room).
- **photos**: room id, slot (corner A, opposite corner, window wall, extra), storage key.
- **designs** and **design_versions**: style, palette, budget, status, render keys; each regeneration creates a version.
- **catalog_items**: title, category (sofa, chair, coffee_table, lighting, rug, wall colour, and others), style tags, price in INR, dimensions, vendor URL, and image.
- **design_items**: design id, catalog item, quantity, placement status, and a reason code if not placed.

## 4.7 Security Considerations

Passwords are stored only as hashes in PostgreSQL. Rooms and designs are scoped to the owning account (“Your rooms and designs stay private to your account”). Object keys are not publicly listable; the frontend receives them through the authenticated API.

# Chapter 5

# Implementation

## 5.1 Authentication and Project Management

Users register and sign in with an e-mail and password (Fig. 6.1). Credentials are verified against PostgreSQL, and subsequent requests carry a session token. A project (“My Flat”) groups rooms (“Living Room”), and every query is filtered by the authenticated user.

## 5.2 Photo Ingestion and Coverage Check

The upload screen offers four slots. The coverage panel reports readiness for a 2D redesign (one photo) and for a 3D model (three photos), and gives capture advice: stand in a corner, shoot toward the opposite one, keeping floor and ceiling in frame. A template photo can be used if the user has none. Photos are stored in MinIO and recorded in PostgreSQL.

## 5.3 Wall and Floor Segmentation

The image worker computes two binary masks per photograph, a wall mask and a floor mask. Fig. 6.5 shows representative intermediate results: for each input photograph the wall mask, the floor mask, the restyled result after recolouring, and the final composite. The wall mask excludes windows, doors, and fixtures, so recolouring does not bleed onto them, and the floor mask both drives floor retexturing and constrains where furniture may appear.

## 5.4 Style and Palette Selection

Only styles with a complete essential catalog set are shown, each with its category counts and a computed minimum budget (Fig. 6.3, Table 5.1). The palette step offers six palettes (Fig. 6.4). The chosen palette defines the wall colour and the floor tone.

Table 5.1: Style catalog coverage and minimum budget (from the running system)

| Style | Sofa | Chair | Coffee tbl. | Lighting | Rug | Min. budget (Rs.) |
|---|---|---|---|---|---|---|
| Bohemian | 1 | 2 | 1 | 2 | 2 | 64,450 |
| Industrial | 1 | 1 | 2 | 3 | 1 | 60,450 |
| Japandi | 1 | 2 | 1 | 3 | 3 | 37,059 |
| Modern Indian Contemporary | 2 | 1 | 2 | 2 | 2 | 74,350 |
| Modern Minimalist | 2 | 2 | 2 | 3 | 2 | 55,020 |
| Nordic / Scandinavian | 2 | 3 | 2 | 2 | 3 | 48,859 |

## 5.5 2D Generation with Stable Diffusion Inpainting

Furniture placement uses the following model configuration:

```
{"model_id": "stable-diffusion-v1-5/stable-diffusion-inpainting",
 "mode": "inpaint", "weight_variant": "fp16",
 "safetensors": "true", "license": "CreativeML OpenRAIL-M"}
```

The inpainting model receives the recoloured room image, a mask covering the target region on the floor, and a prompt composed from the selected catalog item (category, style, colour). Because the model only synthesises content inside the mask, walls, windows, doors, and perspective remain those of the user's photograph. Loading the model in fp16 from safetensors keeps its memory footprint within a T4's 16 GB. Outputs are composited so that unmasked pixels are copied unchanged from the source. The pipeline runs in a Kaggle notebook with a T4 GPU and is exposed to the backend via a Cloudflare endpoint.

Listing 5.1: Simplified inpainting call (illustrative)

```
pipe = StableDiffusionInpaintPipeline.from_pretrained(
   "stable-diffusion-v1-5/stable-diffusion-inpainting",
   torch_dtype=torch.float16, use_safetensors=True).to("cuda")
out = pipe(prompt=item_prompt, image=room_rgb,
        mask_image=placement_mask).images[0]
final = Image.composite(out, room_rgb, placement_mask)
```

## 5.6 Catalog and Budget Engine

For a chosen style and palette, the engine selects one catalog item for each essential category, adds the wall colour as a catalog item (for example, "Wall — Rs. 900"), and sums prices. It enforces three rules:

1. the total must not exceed the user's budget cap;
2. at most six objects are rendered in one design;
3. an item whose rendered size falls below a minimum visible size (16 px) is flagged *Not visible in image* rather than silently counted as shown.

Items beyond the object limit are flagged *Not placed*. In the example in Fig. 6.6, the total is Rs. 64,439 against a budget of Rs. 65,440: sofa-bed Rs. 47,990, chair Rs. 3,990, coffee table Rs. 4,990, pendant lamp Rs. 2,490, rug Rs. 1,490, wall colour Rs. 900, a lint roller Rs. 99, and wall lamp Rs. 2,490. This transparency is how RoomSage avoids asserting an unverified total.

## 5.7 3D Room Preview with Meshy

When three photographs have been uploaded, the 3D model is marked ready. Furniture meshes are produced by sending catalog product images to the Meshy API, and the resulting meshes are placed in a Three.js room (walls, floor, window, door) with orbit controls (Fig. 6.9). COLMAP and Nerfstudio were evaluated for photogrammetric reconstruction of the room shell; the browser-assembled room was retained as the interactive baseline because it is fast, runs on any device, and does not depend on multi-photo overlap quality.

## 5.8 Frontend Implementation

The frontend uses React 19 with Vite. The design workspace overlays numbered markers on the generated image at each item's placement, and the item list uses the same numbers. Items marked not visible or not placed use a dot marker and display their reason. Swap and Remove actions call the API, and the budget bar updates from the server's recomputed total.

## 5.9 Asynchronous Job Handling

Segmentation, inpainting, and Meshy requests can take seconds to minutes. The API returns a job identifier immediately; Celery workers run the task; the frontend polls the status field (the workspace shows "done" next to the design name). This prevents long operations from blocking the UI.

# Chapter 6

# Testing and Results

## 6.1 Test Environment

Table 6.1: Test environment configuration

| Component | Specification |
|---|---|
| GPU worker | Kaggle notebook, NVIDIA T4 (16 GB), CUDA PyTorch |
| 2D model | Stable Diffusion v1.5 Inpainting, fp16, safetensors |
| 3D meshes | Meshy API |
| Backend | FastAPI, SQLAlchemy, PostgreSQL, Celery, Redis, MinIO |
| Frontend | React 19, Vite, Three.js; Google Chrome |

## 6.2 User Workflow Results

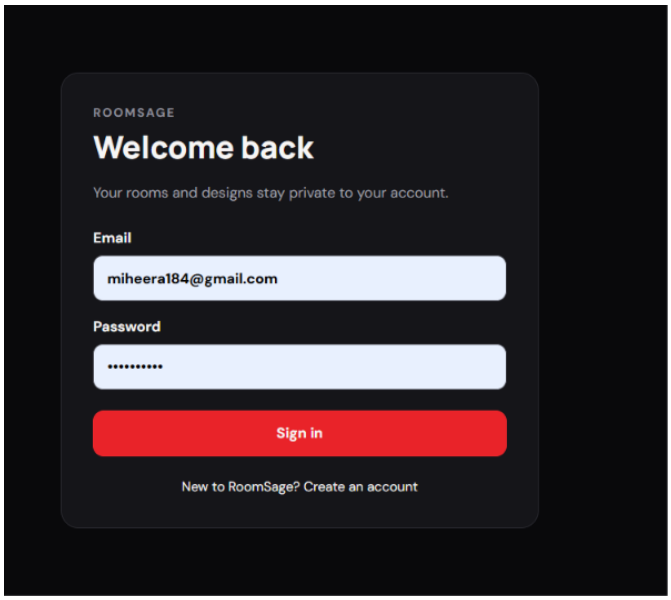



Figure 6.1: Sign-in screen. Rooms and designs stay private to the user's account.

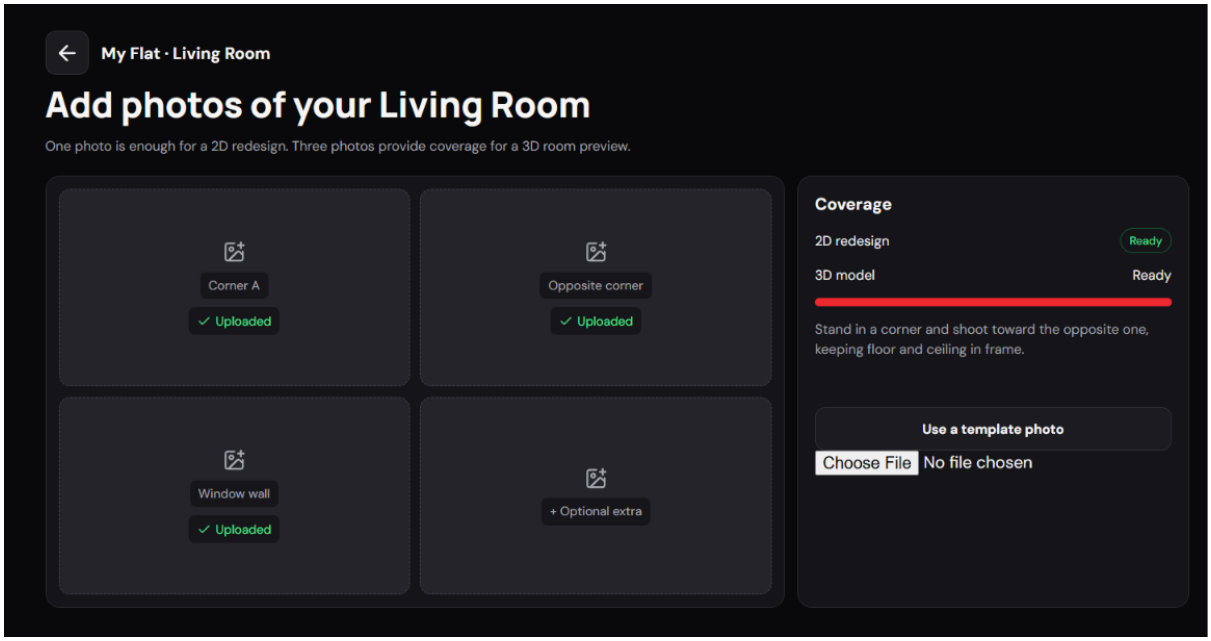



Figure 6.2: Photo upload with Corner A, Opposite corner, and Window wall uploaded; the coverage panel reports 2D redesign ready and shows 3D-model readiness.

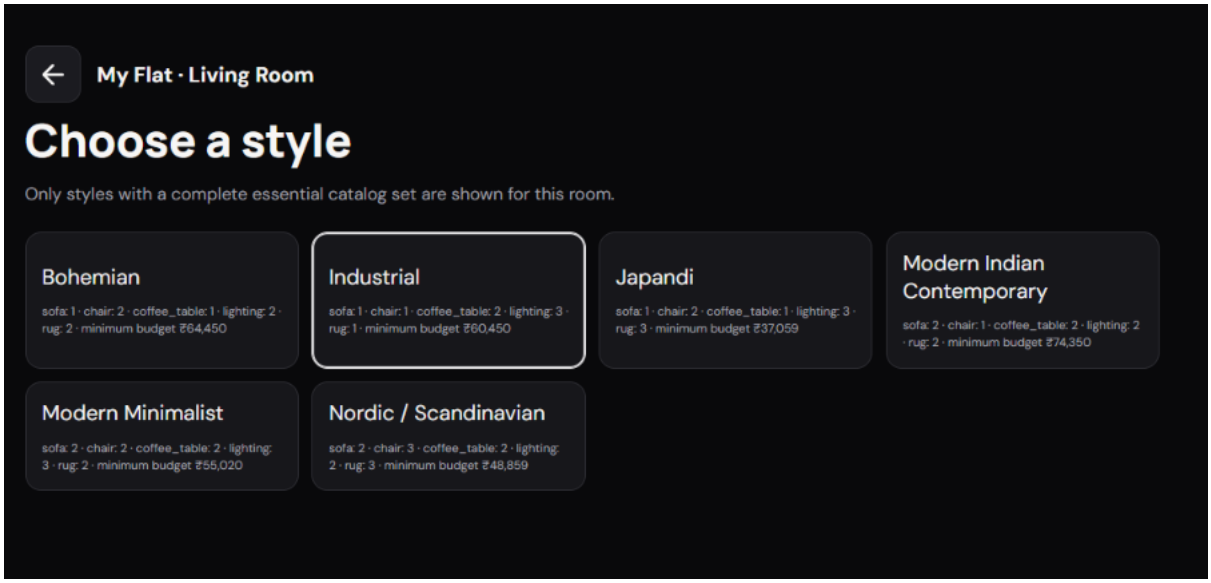



Figure 6.3: Style selection. Only styles with a complete essential catalog set are shown, with category counts and minimum budget.

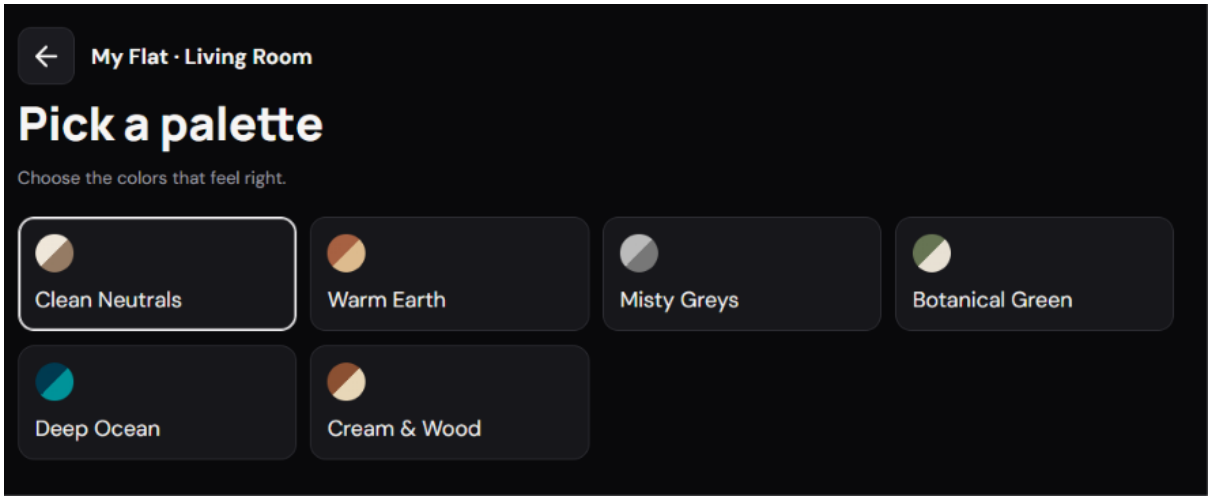



Figure 6.4: Palette selection: Clean Neutrals, Warm Earth, Misty Greys, Botanical Green, Deep Ocean, and Cream & Wood.

## 6.3 Segmentation and Recolouring Results




Figure 6.5: Intermediate pipeline results for six sample photographs. Columns: input, wall mask, floor mask, restyled image, and final image. The masks isolate walls and floors while excluding windows and doors, so the restyling leaves them intact.

Across the samples the wall mask correctly excludes windows, door frames, and ceilings, and the floor mask follows the floor boundary under varied perspectives. Residual artefacts occur at thin structures (door frames, narrow openings), where the wall mask can contain small holes (for example photo_11), and the retextured floor shows a repeating tile pattern in some views.

## 6.4 Furniture Placement Results

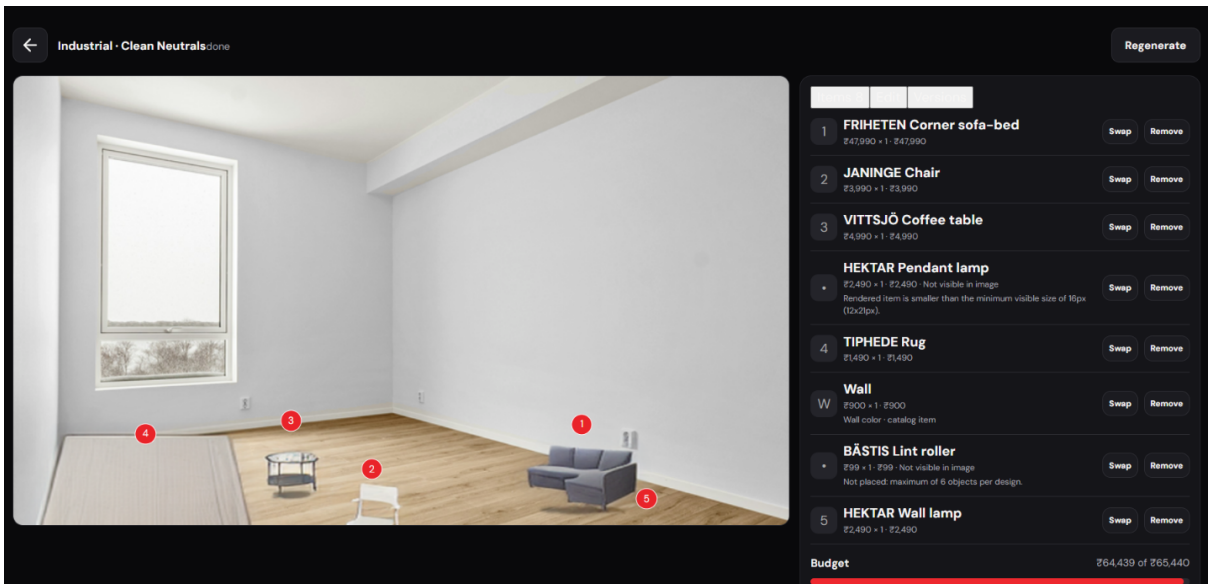



Figure 6.6: Design workspace *before* the diffusion model was trained for the task: furniture is composited as cut-out objects. Items are small, the rug is rendered as a flat slab against the wall, and two items are flagged (pendant lamp not visible, lint roller not placed). The budget is Rs. 64,439 of Rs. 65,440.



Figure 6.7: A 2D generation result for the living-room template: sofa, coffee table, and rug are placed on the floor while the window, walls, and perspective are preserved.



Figure 6.8: Final product of 2D generation (Industrial style, Clean Neutrals palette): the sofa, lounge chair, glass coffee table, and woven rug are lit and scaled consistently with the room, the wall colour follows the palette, and wall art is added.

Comparing Fig. 6.6 and Fig. 6.8, the diffusion-based inpainting yields correctly scaled, shadowed, and perspective-consistent furniture, in contrast with the pre-training composite where objects appear pasted and mis-scaled. In every case the room shell (window, beam, skirting, outlets) is unchanged because those pixels lie outside the mask.

## 6.5 3D Room Preview



Figure 6.9: 3D room preview rendered in the browser: walls, window, door, floor, and a Meshy-generated furniture mesh placed in the room.

## 6.6 Budget Behaviour

Table 6.2 reproduces the itemised plan of Fig. 6.6 and confirms that the sum of line items equals the displayed total.

Table 6.2: Itemised plan for Industrial / Clean Neutrals

| # | Item | Price (Rs.) | Status |
|---|---|---|---|
| 1 | FRIHETEN corner sofa-bed | 47,990 | Placed |
| 2 | JANINGE chair | 3,990 | Placed |
| 3 | VITTSJÖ coffee table | 4,990 | Placed |
| – | HEKTAR pendant lamp | 2,490 | Not visible (below 16 px) |
| 4 | TIPHEDE rug | 1,490 | Placed |
| W | Wall colour | 900 | Catalog item |
| – | BÄSTIS lint roller | 99 | Not placed (max 6 objects) |
| 5 | HEKTAR wall lamp | 2,490 | Placed |
| | **Total** | **64,439** | Budget 65,440 |

## 6.7 Discussion

**What worked.** Mask-restricted inpainting preserved room geometry, the catalog grounding produced a consistent itemised plan, and the budget engine surfaced items that could not be shown rather than hiding them. **Limitations observed.** SD 1.5 inpainting is limited in resolution and may alter furniture detail between runs; thin structures can degrade masks; the T4's memory and the free-tier session limits constrain throughput; and no quantitative user study has yet been performed.

# Chapter 7

# Conclusion and Future Work

## 7.1 Conclusion

RoomSage shows that spatial fidelity and purchasability can be combined in a single, accessible workflow. By segmenting the user's own photograph and inpainting only within masked regions, the system preserves real room geometry; by grounding every item in a priced catalog and enforcing a budget cap, it turns an image into an actionable plan; and by generating meshes with Meshy and rendering them in Three.js, it supports a 3D preview. The full-stack system—React 19/Vite/Three.js, FastAPI/SQLAlchemy/PostgreSQL, Celery/Redis, MinIO, and a CUDA PyTorch image worker on a Kaggle T4—runs on modest, largely free resources.

## 7.2 Future Work

1. Upgrade to SDXL inpainting or depth-conditioned ControlNet for higher resolution and tighter structure.
2. Full photogrammetric/Gaussian-splat reconstruction of the room using COLMAP and Nerfstudio, with Instruct-GS2GS-style 3D edits.
3. Learned layout planning (clearance, traffic paths) and furniture scaling from real dimensions.
4. An LLM planner for natural-language refinement and budget trade-off suggestions.
5. Live vendor integration (stock, price updates, checkout).
6. AR preview, multi-room style continuity, and shared projects.
7. A formal user study measuring perceived fidelity and planning time saved.

# Appendix A

# Installation and Setup Guide

## A.1 Prerequisites

Python 3.10+, Node.js 18+, PostgreSQL, Redis, MinIO, and a Kaggle account (T4 GPU) for the image worker; Meshy and Gemini API keys.

## A.2 Backend and Workers

```
python -m venv venv && source venv/bin/activate
pip install -r backend/requirements.txt
uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
celery -A app.worker worker --loglevel=info
```

## A.3 Frontend

```
cd frontend && npm install && npm run dev
```

## A.4 Image Worker (Kaggle T4)

Run the worker notebook with GPU enabled, install CUDA PyTorch, Diffusers, Transformers, OpenCV, and Pillow, load the inpainting model in fp16, and expose the endpoint through Cloudflare. Set the worker URL in the backend environment.

## A.5 Environment Variables

Table A.1: Environment variables

| Variable | Description |
|---|---|
| DATABASE_URL | PostgreSQL connection string |
| REDIS_URL | Celery broker |
| S3_ENDPOINT, S3_ACCESS_KEY, S3_SECRET_KEY | MinIO storage |
| GPU_WORKER_URL | Cloudflare URL of the image worker |
| MESHY_API_KEY | Meshy 3D generation |
| GEMINI_API_KEY | Gemini API |

# Bibliography


[1] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3D Gaussian Splatting for Real-Time Radiance Field Rendering," *ACM Trans. Graph.*, vol. 42, no. 4, 2023.

[2] L. Zhang, A. Rao, and M. Agrawala, "Adding Conditional Control to Text-to-Image Diffusion Models," in *Proc. IEEE/CVF ICCV*, 2023.

[3] H. Xiang et al., "GaussianRoom: Improving 3D Gaussian Splatting with SDF Guidance and Monocular Cues for Indoor Scene Reconstruction," 2024.

[4] Y. Chen et al., "FreeSplat: Generalizable 3D Gaussian Splatting for Unconstrained Indoor Scenes," in *Proc. NeurIPS*, 2024.

[5] V. Haque et al., "Instruct-GS2GS: Editing 3D Gaussian Splats with Instructions," 2024.

[6] J. Wu et al., "GaussCtrl: Multi-View Consistent Text-Driven 3D Gaussian Splatting Editing," in *Proc. ECCV*, 2024.

[7] Y. Chen et al., "GaussianEditor: Swift and Controllable 3D Editing with Gaussian Splatting," in *Proc. CVPR*, 2024.

[8] D. Tochilkin et al., "TripoSR: Fast 3D Object Reconstruction from a Single Image," arXiv:2403.02151, 2024.

[9] A. Kirillov et al., "Segment Anything," in *Proc. IEEE/CVF ICCV*, 2023.

[10] J. L. Schönberger and J.-M. Frahm, "Structure-from-Motion Revisited," in *Proc. IEEE CVPR*, 2016.

[11] R. Rombach et al., "High-Resolution Image Synthesis with Latent Diffusion Models," in *Proc. IEEE/CVF CVPR*, 2022.

[12] S. Ramírez, "FastAPI," https://fastapi.tiangolo.com.

[13] R. Cabello et al., "Three.js," https://threejs.org.

[14] Meshy AI, "Meshy API documentation," https://docs.meshy.ai.